COER UNIVERSITY

Assignment: Software Testing and Quality Assurance

MCA2nd Year | MCA211D

Submitted by: mahi dhiman ***RollNo:*** 253028050

Q1. What is Software Testing? Explain its objectives and importance.

Definition

Software Testing is the process of executing a program or system with the intent of finding defects and evaluating whether it meets the specified requirements. It checks the actual behaviour of the software against the expected behaviour and provides information about the quality of the product. Testing can be ***manual*** (performed by a tester without tools) or ***automated*** (performed using tools and scripts).

Objectives of Software Testing

- ***Detect defects:*** to find errors, bugs and faults as early as possible, before the software reaches the user.
- ***Verify requirements:*** to confirm that the software does what the Software Requirement Specification (SRS) says it should do.
- ***Build confidence:*** to give stakeholders confidence that the software is reliable and ready for release.
- ***Prevent defects:*** to analyse the causes of defects so that similar mistakes are avoided in future work.
- ***Ensure quality attributes:*** to check functionality, performance, security, usability, compatibility and reliability.
- ***Reduce risk:*** to reduce the risk of failure in operation, which could cause financial loss or harm to users.
- ***Support decision making:*** to supply test results and metrics that help management decide whether to release the product.

Importance of Software Testing

- ***Cost saving:*** a defect found in the requirement or design phase is far cheaper to fix than one found after release.
- ***Quality and reliability:*** tested software behaves consistently and fails less often.
- ***Security:*** testing exposes vulnerabilities such as SQL injection and weak authentication, protecting data.
- ***Customer satisfaction:*** fewer failures lead to a better user experience and stronger trust in the product.
- ***Compliance:*** many domains (banking, healthcare, aviation) require proven testing to meet legal and industry standards.

- ***Brand reputation:*** a serious failure in production can damage a company's image and cause loss of business.

Q2. What is Software Quality Assurance (SQA)? Explain its role in software development.

Definition

Software Quality Assurance (SQA) is a systematic set of planned and preventive activities applied throughout the software development life cycle to ensure that the software processes and the resulting products conform to defined standards and requirements. SQA focuses on the ***process*** used to build software, with the aim of preventing defects rather than only finding them.

Role of SQA in Software Development

- ***Defining and improving processes:*** establishes development and testing procedures and improves them continuously.
- ***Ensuring adherence to standards:*** checks that the team follows standards such as ISO 9001, ISO/IEC 25010, IEEE 730 and CMMI practices.
- ***Defect prevention:*** uses reviews, inspections and audits at every phase to catch problems early.
- ***Monitoring and measurement:*** collects metrics such as defect density, defect removal efficiency and test coverage.
- ***Planning and coordination:*** prepares the SQA Plan, and coordinates with development, testing and management teams.
- ***Reporting:*** provides management with independent visibility into product and process quality.
- ***Risk management:*** identifies quality risks early and recommends corrective action.

Q3. Differentiate between Software Testing and Quality Assurance.

Basis	***Software Testing***	***Quality Assurance***
Meaning	Executing the software to find defects and validate behaviour.	A planned set of activities that ensures the process of development is correct.
Focus	Product oriented.	Process oriented.
Approach	Corrective / detective: finds defects that already exist.	Preventive: aims to stop defects from occurring.
Goal	To identify bugs and confirm requirements are met.	To improve the process so that quality is built in.
Scope	A part of the development life cycle (mostly after coding).	Covers the entire SDLC.
Performed by	Testers / test engineers.	QA team, QA engineers and process managers.

Basis	*Software Testing*	*Quality Assurance*
Activities	Test design, execution, defect reporting, regression.	Audits, reviews, standards, process definition, training.
Type	Considered a Quality Control activity.	Considered a management-level activity.
Example	Running test cases on a login page to check invalid passwords.	Defining a rule that every module must be peer-reviewed before testing.

Q4. Explain the major objectives of Software Quality Assurance.

1. ***Ensure conformance to requirements:*** the product must satisfy the functional and non-functional requirements agreed with the customer.
2. ***Defect prevention:*** identify and remove causes of defects through process control instead of depending only on detection.
3. ***Compliance with standards:*** ensure that recognised standards and organisational policies (ISO, IEEE, CMMI) are followed.
4. ***Process improvement:*** continuously measure, analyse and improve the software development processes.
5. ***Early detection of problems:*** find issues at requirement, design and code stage through reviews and inspections.
6. ***Improve reliability and maintainability:*** ensure software is dependable and easy to modify and support.
7. ***Customer satisfaction:*** deliver a product that meets expectations on time and within budget.
8. ***Visibility and control:*** give management accurate information on project status and product quality.
9. ***Reduce cost and rework:*** lower the cost of fixing defects late in the life cycle.

Q5. What is the difference between Quality Assurance (QA) and Quality Control (QC)?

Basis	*Quality Assurance (QA)*	*Quality Control (QC)*
Definition	Process-oriented activities that ensure quality is built into the product.	Product-oriented activities that verify the finished product meets quality standards.
Objective	Prevent defects.	Identify and correct defects.
Nature	Proactive.	Reactive.
Focus	Processes, methods, standards.	The actual product and its components.

Basis	*Quality Assurance (QA)*	*Quality Control (QC)*
Timing	Throughout the life cycle.	After a work product is produced (mainly testing phase).
Techniques	Audits, process definition, training, reviews, standards.	Testing, inspection, checkpoints, defect tracking.
Responsibility	Entire team, led by QA/process group.	Testing / QC team.
Example	Defining a coding standard and a review process.	Executing test cases and logging bugs found in a build.

Q6. Explain the role and responsibilities of a Software Quality Assurance Engineer.

A ***Software Quality Assurance (SQA) Engineer*** is responsible for making sure that software is developed and tested according to defined quality standards. The engineer monitors the process from requirements to release and acts as an independent guardian of quality.

Responsibilities

- ***Review requirements and design documents*** to check that they are clear, complete, testable and consistent.
- ***Prepare the QA/test strategy and plan,*** defining scope, resources, schedule, tools and risks.
- ***Design test cases and test data*** based on requirements, and maintain traceability between them.
- ***Execute tests*** (manual and automated): functional, regression, integration, performance, security and usability.
- ***Log and track defects*** using tools like JIRA or Bugzilla; report severity and priority; retest fixes.
- ***Develop and maintain automation scripts*** using tools such as Selenium, JUnit or TestNG.
- ***Ensure process compliance*** by conducting reviews, walkthroughs and audits, and checking adherence to standards.
- ***Collect and analyse metrics*** such as defect density, test coverage and defect leakage.
- ***Perform root cause analysis*** of defects and suggest process improvements. ***Communicate***
- ***with stakeholders,*** giving clear status and quality reports, and contribute to release decisions (go/no-go).
- ***Support continuous improvement*** by sharing lessons learned and training team members.

Skills Required

Knowledge of SDLC and testing techniques, analytical and problem-solving ability, attention to detail, familiarity with test and defect-management tools, basic scripting/programming skills, and good communication.

Q7. Explain the relationship between Software Testing, Quality Assurance, and Quality Control.

Software Testing, Quality Assurance and Quality Control are closely related and together form the complete quality effort of an organisation, but they differ in scope.

- ***Quality Assurance (QA)*** is the broadest concept. It is a process-level, preventive activity covering the whole SDLC and establishes the standards, procedures and methods needed to produce quality software.
- ***Quality Control (QC)*** is a subset of the quality effort. It is product-level and detective; it checks that the built product meets the defined quality criteria, using reviews, inspections and testing.
- ***Software Testing*** is the most important technique of QC. It is the actual execution of the software to find defects and to validate the product.

Thus the relationship can be summarised as: ***Software Testing*** $\subseteq$ ***Quality Control*** $\subseteq$ ***Quality Assurance*** (testing is a part of QC, and QC is a part of the overall quality management under QA). QA defines *how* work should be done, QC checks *whether* the outcome is right, and testing is *the way* of checking the software itself. The results of testing and QC (defect trends, metrics) are fed back to QA, which uses them to improve the process, and this creates a continuous cycle of quality improvement.

Q8. What are the different activities performed during Software Quality Assurance?

1. ***Quality planning:*** preparing the SQA Plan, defining quality goals, standards, metrics and responsibilities.
2. ***Defining processes and standards:*** establishing development, coding, documentation and testing procedures.
3. ***Requirements and design reviews:*** checking documents for completeness, clarity and testability.
4. ***Technical reviews, walkthroughs and inspections:*** peer examination of documents and code to find defects early.
5. ***Testing activities:*** planning and monitoring test design, execution and defect tracking.
6. ***Configuration and change management:*** controlling versions and changes so that the product remains consistent.
7. ***Audits:*** independent examination to confirm that processes and standards are actually followed.
8. ***Measurement and metrics collection:*** tracking defect density, defect removal efficiency, coverage, effort and schedule variance.
9. ***Defect management and root cause analysis:*** analysing why defects occurred and preventing them from reoccurring.
10. ***Risk management:*** identifying and mitigating risks that threaten quality.
11. ***Training and mentoring:*** making sure the team understandsthe standards and tools.

12. ***Reporting and process improvement:*** providing status to management and suggesting improvements.

Q9. Explain how Software Testing helps in improving software quality.

- ***Finds and removes defects:*** every defect found and fixed before release directly increases product quality.
- ***Verifies functionality:*** confirms that each feature works as specified in the requirements.
- ***Checks non-functional quality:*** performance, security, usability, compatibility and reliability testing ensure the software is fast, safe and easy to use.
- ***Improves reliability:*** stress and load testing reveal how the software behaves under heavy use so failures in real conditions are reduced.
- ***Regression testing:*** ensures that new changes do not break existing features.
- ***Provides measurable information:*** test coverage, pass/fail rates and defect trends show the actual quality level.
- ***Reduces risk and cost:*** finding defects early avoids expensive production failures and rework.
- ***Feedback for improvement:*** defect analysis shows weak areas in the process and helps prevent recurrence.
- ***Builds user confidence:*** a well-tested product leads to greater trust and satisfaction.

Q10. What is a Quality Management System (QMS)? Explain its importance in software development.

Definition

A ***Quality Management System (QMS)*** is a formalised system that documents the processes, procedures, responsibilities and resources needed to achieve quality objectives and meet customer requirements. In software organisations it is commonly based on ***ISO 9001*** and is often supported by models such as CMMI.

Key Elements of a QMS

- Quality policy and quality objectives.
- Documented processes, procedures and work instructions.
- Defined roles and responsibilities.
- Resource and training management.
- Monitoring, measurement, audits and management review.
- Corrective and preventive action and continual improvement.

Importance in Software Development

- ***Consistency:*** ensures every project follows the same proven processes, so results are predictable.
- ***Customer focus:*** helps meet and exceed customer requirements and expectations.

- ***Reduced defects and rework:*** systematic control lowers errors and saves cost.
- ***Better decision making:*** decisions are based on data and metrics.
- ***Compliance:*** helps to meet legal, regulatory and contractual requirements.
- ***Continual improvement:*** feedback and audits drive constant improvement of processes.
- ***Credibility and competitiveness:*** certification (e.g., ISO 9001) increases customer trust and business opportunities.
- ***Clear accountability:*** roles and responsibilities are well defined.

Q11. Explain the importance of standards, processes, and documentation in Software Quality Assurance.

1. Standards

Standards are agreed rules and guidelines (e.g., ISO 9001, ISO/IEC 25010, IEEE 829, IEEE 730, CMMI, coding standards) that define what is expected of a product or process.

- Provide a common benchmark against which quality can be measured.
- Ensure uniformity and consistency across teams and projects.
- Reduce misunderstandings and improve interoperability and maintainability.
- Help meet regulatory and customer requirements.

2. Processes

A process is a defined sequence of activities for developing, testing and maintaining software.

- Make the results repeatable and predictable rather than dependent on individuals.
- Reduce errors by providing clear steps and control points.
- Enable measurement and hence continuous improvement.
- Allow better planning of time, cost and resources.

3. Documentation

Documentation includes SRS, design documents, test plans, test cases, review records, defect reports, user manuals and audit reports.

- Serves as a record and evidence that processes and standards were followed (needed in audits).
- Improves communication between developers, testers, customers and management.
- Provides traceability from requirements to design, code and test cases.
- Helps in maintenance, training of new team members and knowledge retention.
- Supports analysis of past projects for lessons learned.

Together, standards define ***what*** is required, processes define ***how*** to achieve it, and documentation provides ***proof and knowledge*** that it was done, which forms the base of effective SQA.

Q12. What is a Software Quality Assurance Plan (SQAP)? Explain its major components.

Definition

A ***Software Quality Assurance Plan (SQAP)*** is a formal document that describes the quality activities, standards, resources, schedule and responsibilities that will be used to ensure the quality of a specific software project. It is prepared at the start of the project, generally following the structure of ***IEEE 730***.

Major Components

1. ***Purpose and scope:*** the objective of the plan and the products/processes it covers.
2. ***Reference documents:*** standards and project documents used (SRS, SDD, IEEE, ISO).
3. ***Management:*** organisation structure, roles and responsibilities of the quality team, and tasks to be performed.
4. ***Documentation:*** list of documents to be produced and reviewed (SRS, design, test plan, user manual, etc.).
5. ***Standards, practices, conventions and metrics:*** coding standards, naming rules, and measures that will be collected.
6. ***Reviews and audits:*** types (walkthroughs, inspections, technical reviews, audits), schedule and procedure.
7. ***Testing:*** the approach to test planning, test levels, tools and acceptance criteria.
8. ***Problem reporting and corrective action:*** how defects are recorded, tracked and resolved.
9. ***Tools, techniques and methodologies:*** software used to support quality work.
10. ***Code, media and supplier control:*** configuration control and management of third-party or subcontracted software.
11. ***Records collection, maintenance and retention:*** how quality records are stored.
12. ***Training:*** training needs of the team.
13. ***Risk management:*** identification and handling of quality-related risks.

Q13. Explain the difference between Verification and Validation and discuss their role in software quality.

Verification answers the question *“Are we building the product right?”* It checks that the product is being developed according to the specifications and design. ***Validation*** answers *“Are we building the right product?”* It checks that the final product actually meets the customer’s needs and expectations.

Basis	***Verification***	***Validation***
Meaning	Checks whether the product conforms to specifications.	Checkswhether the product satisfiesuser needs.
Question	Are we building the product right?	Are webuilding the right product?
Type of testing	Static (no code execution).	Dynamic(code is executed).

Basis	*Verification*	*Validation*
Methods	Reviews, walkthroughs, inspections, desk-checking.	Functional, system and acceptance testing, beta testing.
Applied on	Documents, plans, design, code (intermediate work products).	The final working software.
Timing	Before validation, during early phases.	After verification, generally at the end of development.
Performed by	QA team, developers, reviewers.	Testers and often the customer / end user.
Goal	Prevent defects.	Detect defects and confirm suitability.

Role in Software Quality

- Verification catches errors in requirements, design and code early, which reduces cost and prevents defects from spreading.
- Validation confirms that the delivered software works in the real environment and fulfils business needs, which ensures customer satisfaction.
- Together (as in the V-Model, where each development phase has a matching test phase) they provide complete assurance of both correctness and fitness for purpose.

Q14. Explain how defects can affect the overall quality of software.

A ***defect*** (bug) is a flaw in the software that causes it to behave differently from what is expected. Defects reduce quality in several ways:

- ***Functional failures:*** features do not work correctly, so the user cannot complete tasks.
- ***Poor reliability:*** crashes, hangs and data corruption reduce dependability.
- ***Performance problems:*** slow response, memory leaks and poor scalability degrade the user experience.
- ***Security risks:*** vulnerabilities allow unauthorised access, data theft or fraud.
- ***Usability issues:*** confusing or inconsistent behaviour frustrates users.
- ***Higher cost and delay:*** fixing defects late requires rework, retesting and redeployment, which increases budget and schedule.
- ***Lower maintainability:*** many defects make code harder to understand and modify, and fixes may introduce new bugs.
- ***Loss of customer trust and reputation:*** repeated failures lead to complaints, customer loss and negative reviews.
- ***Legal and financial consequences:*** in critical domains (banking, healthcare, aviation), defects can cause financial loss, penalties or even risk to human life.

Therefore, minimising defects through prevention, early detection and effective testing is essential for high-quality software.

Q15–16. A banking application frequently crashes during online transactions. As a QA professional, explain what activities you would perform to improve the quality of the application.

Since a banking application deals with money and sensitive data, frequent crashes are a serious problem. I would follow a systematic approach:

1. ***Understand and reproduce the problem:*** collect crash reports, logs, screenshots and user complaints; identify the transaction type, device, OS, network and load conditions in which the crash occurs; reproduce the issue in a test environment.
2. ***Defect logging and prioritisation:*** raise defects in a tracker with severity ***Critical*** and high priority, and give clear steps to reproduce.
3. ***Root cause analysis:*** work with developers to analyse logs, stack traces, database and server behaviour to find whether the cause is a coding error, memory leak, database deadlock, timeout, concurrency issue or server capacity.
4. ***Review requirements and design:*** check whether transaction requirements (e.g., session handling, rollback, error handling) were properly specified and designed; conduct code reviews and inspections of the transaction module.
5. ***Strengthen test design:*** add test cases for normal, boundary and negative scenarios (insufficient balance, invalid input, session timeout, network drop midway, duplicate submission).
6. ***Functional and integration testing:*** test the complete transaction flow, including integration with payment gateway, core banking system, SMS/email service and third-party APIs.
7. ***Performance, load and stress testing:*** simulate thousands of concurrent users with tools such as JMeter or LoadRunner to find the breaking point and bottlenecks.
8. ***Security testing:*** verify authentication, encryption, session management and protection from attacks such as SQL injection and XSS.
9. ***Compatibility testing:*** test on different browsers, mobile devices, OS versions and screen sizes.
10. ***Transaction integrity testing:*** check ACID properties, that money is not lost or duplicated when a crash happens, and that rollback/recovery works.
11. ***Regression testing and automation:*** after fixes, run automated regression suites so that fixes do not break other features; integrate them into the CI/CD pipeline.
12. ***Monitoring and process improvement:*** implement production monitoring and alerting, track metrics (crash rate, defect leakage, response time), update checklists and standards, and conduct post-mortem to prevent recurrence.
13. ***Release control:*** use staged rollout (UAT, pilot, then full release) with a rollback plan.

Result: fewer crashes, secure and reliable transactions, and improved customer confidence in the banking application.

Q17. Explain the importance of continuous improvement in Software Quality Assurance.

Continuous improvement means constantly reviewing and refining processes, methods and tools to make software development and testing more effective. It is based on models such as ***PDCA (Plan–Do–Check–Act)***, ***Kaizen***, ***Six Sigma*** and ***CMMI*** maturity levels.

Importance

- ***Adapts to change:*** technologies, customer needs and market conditions change constantly, so processes must evolve.
- ***Reduces defects over time:*** root cause analysis and lessons learned prevent recurring errors.
- ***Improves efficiency and productivity:*** removing waste and automating repetitive tasks saves time and cost.
- ***Increases customer satisfaction:*** better quality with each release builds customer trust.
- ***Competitive advantage:*** organisations that improve continuously deliver faster and with higher quality.
- ***Employee learning and motivation:*** teams gain skills and take ownership of quality.
- ***Higher process maturity:*** helps the organisation progress through CMMI levels towards optimised processes.
- ***Data-driven decisions:*** metrics and feedback guide improvement rather than guesswork.

Q18. What are the major challenges faced while implementing Software Quality Assurance in an organization?

1. ***Lack of management support:*** quality is seen as a cost, so adequate budget and authority are not provided.
2. ***Resistance to change:*** employees may resist new standards, reviews and audits, considering them extra work.
3. ***Time and schedule pressure:*** tight deadlines lead teams to skip reviews and testing.
4. ***Limited resources:*** shortage of skilled QA staff, tools and test environments.
5. ***Unclear or changing requirements:*** frequent changes make it hard to plan and test properly.
6. ***Lack of skills and training:*** team members may not know standards, techniques or tools.
7. ***Poor communication:*** gaps between developers, testers, customers and management cause misunderstandings.
8. ***Difficulty in measuring quality:*** quality is intangible, and choosing meaningful metrics is hard.
9. ***Inadequate documentation:*** missing or outdated documents make verification difficult.
10. ***High cost of implementation:*** certification, tools and training need investment with benefits that are seen only in the long term.

11. ***Tool and technology selection and integration:*** choosing and integrating automation tools with existing systems is complex.
12. ***Distributed teams and outsourcing:*** maintaining uniform quality across locations and vendors is difficult.
13. ***Rapid technology change (Agile, DevOps, Cloud, AI):*** processes and skills must be updated constantly.

Q19. Explain the role of reviews, inspections, and audits in Software Quality Assurance.

Reviews, inspections and audits are the main ***static quality activities*** of SQA. They examine work products and processes without executing the code, and find problems early.

1. Reviews

A review is an examination of a work product (SRS, design, code, test plan) by the author's peers or stakeholders to detect defects and to improve quality. Types include informal reviews, ***walkthroughs*** (author leads the team through the document), and ***technical reviews*** (experts evaluate technical suitability against standards).

- Detect ambiguity, omissions and errors in requirements and design early.
- Encourage knowledge sharing and team understanding.
- Reduce cost of fixing defects.

2. Inspections

An inspection (e.g., ***Fagan inspection***) is the most formal type of review, led by a trained moderator with defined roles (author, reader, recorder, inspectors) and a documented procedure: planning, overview, preparation, inspection meeting, rework and follow-up.

- Very effective at finding defects; often finds a large share of the errors present in a work product.
- Uses checklists and entry/exit criteria, and produces measurable data.
- Improves the development process through defect-cause analysis.

3. Audits

An audit is an independent, systematic examination of processes and work products to determine whether they comply with plans, standards and contractual requirements. Audits may be internal (by the organisation's own QA group) or external (by a customer or certification body).

- Verify that defined processes and standards are actually followed.
- Identify non-conformances and demand corrective actions.
- Provide objective evidence for certification (ISO, CMMI) and give assurance to management and customers.

Aspect	*Review*	*Inspection*	*Audit*
Formality	Informal to semi-formal.	Highly formal.	Formal and independent.
Purpose	Find defects, share knowledge.	Find defects using a strict procedure.	Check compliance with standards and processes.
Conducted by	Peers / team.	Trained inspection team with moderator.	Independent auditors.
Object	Work product.	Work product.	Processes and products.

Q20. Explain how QA activities are performed throughout the Software Development Life Cycle (SDLC).

QA is not restricted to the testing phase; it is applied in ***every phase*** of the SDLC, as follows:

SDLC Phase	*QA Activities*
1. Requirement Analysis	Review the SRS for completeness, clarity, consistency and testability; identify ambiguities; prepare the requirement traceability matrix (RTM); define acceptance criteria; prepare the SQA plan and test strategy.
2. Design	Review high-level and low-level design; check design against standards; verify that all requirements are covered; identify risks; start test case design and test data planning.
3. Implementation / Coding	Enforce coding standards; conduct code reviews and inspections; support unit testing; use static analysis tools; control versions through configuration management.
4. Testing	Execute integration, system, regression, performance, security and usability tests; log and track defects; monitor test coverage and metrics; verify fixes; perform User Acceptance Testing support.
5. Deployment / Release	Check release readiness (exit criteria); perform smoke and sanity tests; verify deployment procedures and documentation; do configuration audits; give go/no-go input.
6. Maintenance	Perform regression testing for every patch and enhancement; analyse production defects and customer feedback; conduct root cause analysis; update documents; feed lessons learned into process improvement.

In this way, QA works as a continuous, preventive activity across all phases, and the V-Model shows this well because each development phase is linked with a corresponding verification or testing activity.

Q21. Explain with a suitable real-world example how effective Software Testing and Quality Assurance can improve customer satisfaction.

Example: An E-commerce Application (Online Shopping Platform)

Consider an online shopping company that is launching its app before a big festival sale, when lakhs of customers will shop simultaneously. The company follows strong QA and testing practices. ***QA activities (prevention)***

- Requirements are reviewed so that features such as search, cart, coupons, payment and order tracking are clear and testable.
- Coding standards and code reviews are enforced, and a defined release process is followed.
- Audits confirm that the team follows the agreed test and release procedures.

Software testing activities (detection)

- ***Functional testing:*** search, add-to-cart, coupon application and checkout work correctly.
- ***Performance and load testing:*** the system is tested with several times the expected traffic so that it does not crash during the sale.
- ***Security testing:*** payment details and personal data are protected.
- ***Compatibility and usability testing:*** the app works smoothly on different phones, browsers and screen sizes, and the interface is easy to use.
- ***Regression testing:*** new offers and features do not break the existing ones.
- ***Payment failure testing:*** in case of network drop, money is not deducted twice and refunds work properly.

Impact on customer satisfaction

- ***Reliability:*** the app remains available and fast even during peak load, so customers can complete purchases.
- ***Correctness:*** correct prices, discounts and order details prevent disputes and complaints.
- ***Security and trust:*** customers feel safe about sharing their card and address information.
- ***Smooth experience:*** an easy, bug-free interface encourages repeat purchases and positive ratings.
- ***Lower support cost:*** fewer complaints and returns reduce the load on customer care.
- ***Brand loyalty:*** satisfied customers recommend the platform, which increases the customer base and revenue.

If the same company skipped testing and QA, customers would face crashes, wrong charges and slow pages during the sale, leading to lost sales, bad reviews and lost trust. This example shows that ***effective testing and QA directly translate into higher customer satisfaction, loyalty and business success.***